Available online at www.sciencedirect.com

ScienceDirect

Procedia CIRP 119 (2023) 1059–1064

www.elsevier.com/locate/procedia

33rd CIRP Design Conference

Benchmark of the Physics Engine MuJoCo and Learning-based Parameter Optimization for Contact-rich Assembly Tasks

Balázs András Bálint*,[a], Arik Lämmle[a], Yannis Salteris[a], Philipp Tenbrock[a]

[a]*Fraunhofer Institute for Manufacturing Engineering and Automation IPA, Nobelstraße 12, Stuttgart 70569, Germany*

* Corresponding author. *E-mail address:* balazs.andras.balint@ipa.fraunhofer.de. All authors contributed equally.

Abstract

Digital simulation tools are used throughout the entire product development cycle, from designing components using finite element methods to simulating their assembly and planning their production. Especially in research, physics simulators and engines such as MuJoCo are increasingly used for rapid data generation in training machine learning models. The quality of the training data is crucial for successful training and the subsequent deployment of the models in real-world applications. The presented work reports on a systematic benchmark of the physics engine MuJoCo. Representative processes in robotic assembly applications are carried out in simulation and on a physical testbed. The results are compared, focusing on the robot arm's pose and the acting contact forces during the assembly processes using Minimal Jump Cost. Subsequently, a deep reinforcement learning agent takes over the simulation's parameterization, and a first comprehensive system for the process-specific training of simulation parameters is developed and evaluated to increase the data quality and to partially encapsulate the setup effort involved in digital simulation, aiding offline design and validation. The approach results in overall 8% less dissimilar simulations but not in all the individual scenarios.

© 2023 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer review under the responsibility of the scientific committee of the 33rd CIRP Design Conference

Keywords: Physics Simulation MuJoCo; Assembly Automation; Deep Reinforcement Learning

1. Introduction

The mass customization paradigm remains relevant and desirable in today's industry. The need for high-quality customer-individual goods results in production batch sizes down to one; therefore, highly adaptable and efficient production systems and processes are desirable to provide these goods with a competitive price tag with mass-produced ones. Robotic systems are potentially adaptive enough [1, 2], and there is unexploited potential in industrial robotic automation; however, the effort involved in (re)programming robotic systems prohibits their application for batch sizes close to one [3, 4].

Our parallel work [5–8] explores the potential of force-controlled robot programs trained offline on synthetic data from a physics engine to eliminate this obstacle of programming. For such an approach to be viable, the correspondence between generated synthetic data, usually a force signal, and the physical reality, the sim2real gap [9], is crucial.

However, there is yet to be a definitive best simulator or physics engine currently available. Finding relevant comparisons is challenging as physics simulation remains a field of active research, and the relevant tools constantly develop, potentially making any comparison study quickly outdated. The one by Chung et al. [10] is among the most recent, and the examined engines have changed significantly since then. Nonetheless, they find and corroborate that tuning the physics engines' parameters is crucial to achieving their best performance, which remains apparent also in the findings of Mouret et al. [11]. The latter find that none of the examined engines could achieve significantly better overall performance regardless of tuning, suggesting the need for a superior overall simulation tool. No more recent study would refute the idea, whereas further similar work [12–14] suggests that tuning one's simulation remains a suitable method of increasing accuracy on a per-use-case basis.

Nevertheless, tuning the parameters of simulators or physics engines requires expert knowledge and effort, especially to handle unmodeled dynamics [15]. There is generally a discrepancy between data available from producers of industrial hardware and the parameter values a physics simulation expects, e.g., suitable stiffness, damping, and controller gain values are rarely

2212-8271 © 2023 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer review under the responsibility of the scientific committee of the 33rd CIRP Design Conference
10.1016/j.procir.2023.03.149

available readily. At the very least, working with digital simulations requires expert knowledge in the fields of, e.g., digital modeling and control theory, and the task itself is generally non-value-adding from a production perspective. Deep reinforcement learning algorithms are promising candidates for a first solution to handle the complexity involved, as they are shown to be able to solve complex problems and provide model-free solutions [16].

In this work, we propose and examine an approach to encapsulate the effort and knowledge required to (re)tune physics simulations, facilitating and enabling the offline design of robotic solutions with a focus on their force-controlled robot program. We define a set of benchmark processes relevant to force-controlled robotic applications to serve as a platform for qualifying physics simulations and implement these processes on a real-world robotic test bed. Then, we utilize these processes to examine the out-of-the-box performance of the physics engine MuJoCo [17] and set a benchmark by applying the time series dissimilarity metric Minimal Jump Cost [18] to the real-world and synthetic data. Finally, we introduce a deep reinforcement learning approach to optimize the performance of the benchmark processes' simulations on the dissimilarity metric to encapsulate the expert knowledge required for the parameter tuning and examine its performance.

This paper is structured as follows: Section 2 describes the benchmark processes and introduces the real-world test bed; Section 3 defines the machine learning problem for optimizing the simulations; and Section 4 reports the simulations' out-of-the-box and optimized performances and discusses the findings. Finally, Section 5 summarizes our conclusions and depicts the outlook for our further work.

2. Benchmark

To train a force-controlled robot control policy offline in a physics simulation [5], the synthetic force signal describing the robot's interaction with the environment and the synthetic data representing the robot's kinematic state are both crucial. Therefore, we define the benchmark processes focusing on these two signals to gain insight into the physics simulation's ability to generate them. In this work, we use a UR10e 6-degree-of-freedom robot arm for manipulation, which we program through pitasc [19] by defining a sequence of robot movements, i.e., skills in the feature domain. This Section describes the benchmark processes, the real-world testbed used to record the ground truth data, and the structure of the data recorded.

2.1. Robotic Processes

The peg-in-hole task is a benchmarking process of solutions with force-based control that is widely used [20–23] as it is at the core of many industrial processes. The task's setup consists of a body with a hole (hole body) and a peg. To solve the peg-in-hole task, one must localize both bodies, then funnel and insert the peg into the hole. The challenge generally lies in overcoming the tolerances and the localizations' uncertainties.

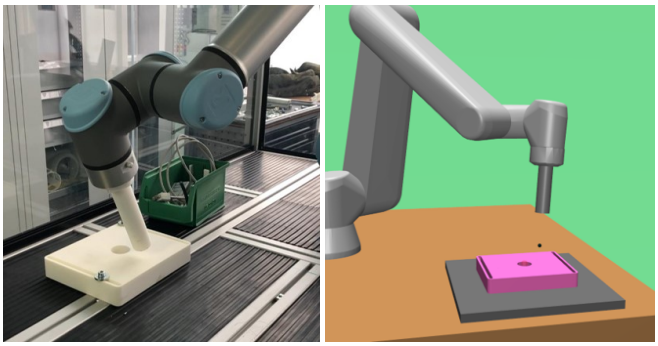


Fig. 1. The real-world test bed and its virtual replica

Fig. 1 shows the present work's robotic setup, the same as in our parallel endeavor [5]; the peg and hole bodies are printed through stereolithography with diameters of 39 mm and 40 mm, respectively. For manipulation, we use a UR10e 6-degree-of-freedom robot arm with the peg rigidly attached, thus, already localized. The manipulator is programmed through pitasc [19] by defining a sequence of robot movements, i.e., skills in the feature domain; in the present work, we use three skills from the solution proposed in [5]. First, from the peg's initial pose above the hole body, the (i) *guarded approach* drives the peg downwards in the negative z-direction until it reaches the hole body and a contact force is measured. The consequent two *guarded slides* move the peg along the hole body's surface while keeping the established contact force at a target value. The (ii) *slide to hole* moves the peg body to the center of the hole. In contrast, the (iii) *slide excentric* moves the peg so that it moves above the hole but misses its center. The two slide skills represent the possible outcomes of the hole's localization; the skill slide to hole emulates success, and the slide excentric failure.

Data is recorded from executing the three skills with varying process parameters to provide a diverse dataset for benchmarking. Table 1 shows the 15 processes used. The parameter v is the velocity of the Cartesian movement of the robot arm's end-effector. The parameter F is either the lower bound of the force threshold for the guarded approach to trigger its success or is the target contact force given to the force controller for pushing against the surface of the hole body while executing sliding during a guarded slide. The parameter β is the angle by which the peg is tilted from its perpendicular alignment with the hole body. Finally, the parameter α is the angle of the starting point of the slide movements on a radial pattern around the hole center; the purpose of varying α is to use a broader range of the manipulator's joints and to vary if it is pushing or pulling the peg along the hole body.

2.2. Recording the Ground Truth

The force signal measured at the flange of the manipulator is recorded for all 15 processes shown in Table 1 in two time series with a frequency of 500 Hz. The series *z-force* contains the magnitude of the force acting along the z-axis of the force-torque sensor. The series *xy-force* contains the vector product of the force components acting along the x- and y-axes, i.e., the

Table 1. The skills and their parameter configurations as used in the present work

	Skill	$v; \frac{m}{s}$	$F; N$	$\beta; °$	$\alpha; °$
1	slide excentric	0.0075	5	15	45
2	slide excentric	0.01	5	15	180
3	slide excentric	0.0075	12.5	15	45
4	guarded approach	0.015	5	5	
5	guarded approach	0.025	20	5	
6	guarded approach	0.03	12.5	5	
7	slide to hole	0.0075	12.5	15	90
8	slide to hole	0.0125	20	15	30
9	slide to hole	0.0125	12.5	15	255
10	slide excentric	0.0125	5	15	135
11	slide excentric	0.01	20	15	135
12	guarded approach	0.025	20	25	
13	guarded approach	0.005	5	25	
14	slide to hole	0.0125	12.5	15	330
15	slide to hole	0.0125	5	15	0

Table 2. In detail table of the performance of a reference simulation throughout the different processes

	avg. MJC	$F_{z,r}, N$	$F_{z,s}, N$	$F_{xy,r}, N$	$F_{xy,s}, N$
1	277.411	17.833	10.824	20.333	2.854
2	6.080	9.720	11.644	4.490	2.824
3	271.637	20.904	17.707	20.615	5.981
4	5.575	12.239	15.170	1.816	6.736
5	43.732	34.257	25.187	6.390	11.146
6	5.409	29.928	23.159	5.807	6.737
7	209.222	19.104	17.217	21.573	5.947
8	529.526	31.748	25.383	38.513	8.317
9	2.229	23.695	19.319	7.491	5.334
10	4.282	18.951	12.032	8.833	3.111
11	190.705	34.910	24.783	24.377	8.663
12	8.165	23.388	26.144	3.419	8.959
13	0.352	6.518	6.889	1.274	2.951
14	3.472	31.901	19.483	18.084	5.381
15	1643.191	13.944	11.520	27.058	3.320

product of the two values recorded by the sensor. The joint positions and velocities are also recorded to represent the manipulator's kinematic state. By recording the data from the process executions on the real-world test bed and in a baseline digital simulation, we establish a ground truth for the simulation performance. For the baseline simulation, the MuJoCo model of the test bed is created; then, the real-world processes are replicated in the virtual environment with the physics engine's out-of-the-box configuration.

2.3. Dissimilarity Metric

Different dissimilarity metrics are used to quantify the correspondence of time series in robotics research and other areas, e.g., speech recognition. Serrà and Arcos [24] investigated the accuracy of the most common (dis)similarity measures. They concluded that the metrics Time Warp Edit Distance [25], Dynamic Time Warping [26], and Minimum Jump Cost (MJC) [18] are the most accurate. On top of accuracy, the runtime performance of the metrics is essential in the current work for the subsequent parameter optimization (Section 3); we performed and timed 10 000 calculations with each metric, and Table 3 shows the average times. We find that the metric MJC required the least computation time; therefore, we use it to quantify the dissimilarity of the real-world and simulated force time series. Table 2 shows the reference simulations' performance on the metric MJC.

3. Parameter Optimization

We frame the optimization problem as an OpenAI Gym [27] environment to exploit the interface and its connection to a wide range of optimizers. In the current work, the agent's action means (i) instantiating the test bed's digital model and the physics engine MuJoCo with a set of parameters, (ii) randomly selecting and executing one of the processes defined in Table 1 in simulation while recording the time series z-force and xy-force as introduced in Section 2.2.

3.1. Action

We expose a set of MuJoCo model parameters through action vectors for the agent to optimize. The action vector $\mathbf{a}_1$ consists of manipulator-specific parameters

$$\mathbf{a}_1 = \langle k_1, \ldots, k_6, b_1, \ldots, b_6, c_1, \ldots, c_6 \rangle , \tag{1}$$

where k_i is the joint stiffness, b_i is the joint damping, and c_i is the joint friction loss for each joint i of the manipulator. In our previous and parallel work, these parameters are generally ones for which sufficiently accurate values are hard to come by as manufacturers rarely provide them; even if they do, the values might still not fulfill the modeling requirements for a specific problem.

3.2. Reward

The reward the agent gets for its actions consists of two components

$$r = 0.8\, r_{MJC} + 0.2\, r_{forces}, \tag{2}$$

where r_{MJC} is a reward based on the dissimilarity metric, and r_{forces} is a reward based on the difference in the maximum forces occurring in the recorded time series. The former is defined as

$$r_{MJC} = \begin{cases} -\frac{1}{150}\Delta MJC + 1, & if\ \Delta MJC < 150 \\ -1, & if\ \Delta MJC \geq 150 \end{cases}, \tag{3}$$

with

$$\Delta MJC = MJC_{baseline} - MJC_{tuned}, \tag{4}$$

where $MJC_{baseline}$ is the dissimilarity calculated between the forces recorded in the baseline simulation and those from the real-world test bed, while MJC_{tuned} is the dissimilarity between the simulation tuned by the agent's action and the real world. The value 150 in the coefficient and the conditional part of Eq. 3 results from hyperparameter tuning.

The reward component that comes from the force values is defined as

$$r_{forces} = 0.5\, r_{forces,z} + 0.5\, r_{forces,xy}, \tag{5}$$

with the components as

$$r_{forces,i} = \begin{cases} 1.00, & \textit{if} \quad 0.05 > e_{forces,i} \\ 0.75, & \textit{if} \quad 0.10 > e_{forces,i} \geq 0.05 \\ 0.50, & \textit{if} \quad 0.15 > e_{forces,i} \geq 0.10 \\ 0.25, & \textit{if} \quad 0.20 > e_{forces,i} \geq 0.15 \\ -1.00, & \textit{otherwise} \end{cases}, \tag{6}$$

with

$$e_{forces,i} = \frac{\left|\max\{F_{i,real}\} - \max\{F_{i,simulated}\}\right|}{\max\{F_{i,real}\}}, \tag{7}$$

where $\max\{F_{i,j}\}$ is the maximum value in the time series corresponding to the force axes $i \in \{z, xy\}$ that was recorded from the $j \in \{real, simulated\}$ world. This latter reward component r_{forces} is introduced to prevent the agent from developing a policy that exploits temporal instabilities in the simulation.

3.3. Observation

We define the observation vector $\mathbf{o}$ the agent can use as input for its policy to determine its following action as

$$\mathbf{o} = \langle m_1, \ldots, m_{20} \rangle, \tag{8}$$

where the values m_i are an equidistant history of the dissimilarity metric calculated during the previous action, i.e., process execution.

The chosen dissimilarity metric MJC is a cumulated one. It is calculated over every data point pair in the two time series it compares, then aggregated through summation. We record all these intermediate values during the process executions, then extract 20 equidistant samples from them and provide these 20 values as observation to the agent. This way, the agent is aware of the development of the dissimilarity, not just the overall quality as one scalar value.

Table 3. The times required by the dissimilarity metric for their calculations

Metric	Time per calculation, s
Time Warp Edit Distance	0.239 54
Dynamic Time Warp	0.030 78
Minimal Jump Cost	0.016 87

3.4. Agent

For an optimizer, we use the synchronous, deterministic variant (A2C) [28] of the Asynchronous Advantage Actor-Critic algorithm as implemented in the library Stable Baselines3 [29]. Related research shows that A2C produces robust and stable learning [30, 31]. It performed marginally the best compared to the Soft Actor-Critic (SAC) [32] and the Proximal Policy Optimization (PPO) [31] on the current problem. These three algorithms are the ones that are established enough for Stable Baselines3 to provide them and are compatible with the problem's action space. The learning rate η of the algorithm A2C is set to

$$\eta = 0.001, \tag{9}$$

which results from hyperparameter tuning over the values $\{0.01, 0.001, 0.0001\}$. We use the out-of-the-box values for all other hyperparameters, and the agent's training consists of 30 000 epochs, each containing one action step. We separate the processes in Table 1 into two groups; processes 1 to 9 are used during training to generate experience for the agent, and processes 10 to 15 are kept back and used only during evaluation to the generalization capability of the agent's policy.

4. Results

4.1. Benchmark

Fig. 2 shows a pair of force time series recorded from the real-world test bed and their counterpart from the reference out-of-the-box simulations per Section 2. Table 2 summarizes the reference simulations' performance on the dissimilarity metric MJC.

4.2. Parameter Tuning

The trained policy produces simulations with an average MJC of 137.958 over all the training processes (Table 4) compared to the reference simulations' 150.091; the optimized simulations can reproduce the real-world data by 8.08% better according to the dissimilarity metric. Process-wise, the optimized simulations can produce data less dissimilar to reality in 5 out

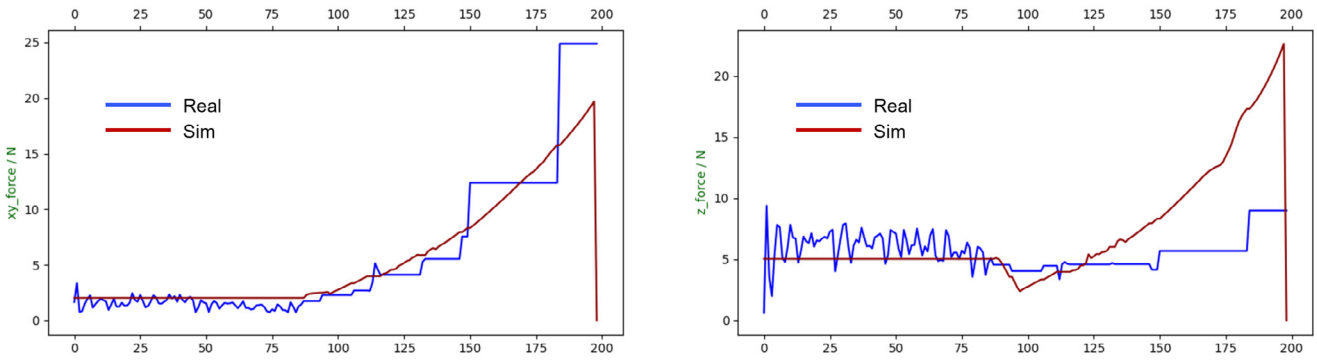


Fig. 2. A pair of force time series as recorded for the test bed and generated from simulation with the reference setup

Table 4. Performace of the trained policy on the dissimilarity metric per process

	MJC_{ref}, -	MJC_{opt}, -	delta, %
1	277.411	208.926	−24.69
2	6.080	11.467	88.60
3	271.637	114.363	−57.90
4	5.575	0.963	−82.72
5	43.732	48.718	11.40
6	5.409	4.019	−25.69
7	209.222	92.571	−55.75
8	529.526	692.298	30.74
9	2.229	68.295	2963.92
mean	150.091	137.958	−8.08
10	4.282	1.277	−70.18
11	190.705	60.185	−68.44
12	8.165	9.759	19.52
13	0.352	1.773	403.83
14	3.472	2.573	−25.88
15	1643.191	58.320	−96.45
mean	308.361	22.315	−92.76

of the 9 cases; they produce MJC dissimilarities in the range of [0.963, 692.298] compared to the references' [2.229, 529.526]. Over the processes the agent does not encounter during training, the trained policy produces simulations with an average MJC of 22.315 (Table 4). In contrast, the reference simulations perform with an MJC of 308.361, which means that the optimized simulations can better reproduce real-world data by 93.00% according to the dissimilarity metric. Process-wise, the optimized simulations can produce data less dissimilar to reality in 4 out of the 6 cases; they produce MJC dissimilarities in the range of [1.773, 60.185] compared to the references' [0.352, 1643.191].

4.3. Discussion

It first stands out when looking at the results of the training processes (Table 4) that the optimized simulations are not consistently less dissimilar than the out-of-the-box ones used as references, despite the overall improvement. For example, for the processes using the skill *guarded approach* (Table 1), the simulations for processes 4 and 6 are improved by 82.72% and 25.69%, respectively, whereas the optimized simulation for process 5 was 11.40% more dissimilar to reality than the corresponding reference. The same is observable for the processes using the skill *guarded slide*; the trained policy improves the simulations for processes 1, 3, and 7 while worsening them for 2, 8, and 9. There appears to be no intuitive pattern in this variance; the changes in the process parameters, i.e., the magnitude of target forces and velocities, do not explain the changes in the performance.

The variance in the performance might be partially attributed to how the OpenAI Gym environment selects the process for which the agent configures the simulation by its action. During both training and evaluation, a random one is sampled. This selection method potentially results in the agent encountering the processes not uniformly, introducing a bias to the observed information. Aggregating the rewards over several simulations per action should compensate for this bias, but developing a custom method of selection that eliminates it would be desirable; e.g., such a selector could be an iterator that steps through each available process. Also, a more thorough hyperparameter tuning could bring different results. For example, the structure of the agent's deep neural network policy and the reward function holds excellent promise. For the latter, a more thorough tuning of the limit of the current reward r_{MJC} (Eq. 3) is interesting, and considering an overall continuous function could help the optimizer.

Admittedly, the current approach is a first take on automating simulation tuning and assumes that there should be an optimal black-box solution. In contrast, it might be worth articulating the problem more by exploiting formal expert knowledge from digital modeling and signal processing, e.g., qualifying the synthetic data in the frequency domain or along existing stability concepts. Developing a more complex metric that quantifies significant aspects of the data instead of the domain-agnostic MJC could improve the optimization.

5. Conclusion

This paper presented a set of processes relevant to the field of force-controlled robotic solutions for use in benchmarking

physics simulations. First, the processes were implemented in a real-world setup; the joint positions were recorded while executing the processes to describe the kinematic state and a force signal for the dynamic. Then, a benchmark was set for the widely adopted physics engine MuJoCo by implementing the same processes and quantifying the dissimilarity between the force data from the real world and the simulated using the metric Minimum Jump Cost. Finally, a machine-learning approach was proposed to optimize the synthetic data quality by tuning the physics engine parameters, and the training results were reported. The approach resulted in around 8% less dissimilar data being generated, though the improvement was not present and consistent per process. Future work on the subject should include a better process selection scheme during the agent's training and the evaluation of its performance. Also, a more in-depth examination of hyper-parameters and the generalization capability of the tuned simulation parameters in a broader range of processes would be fascinating; but an endeavor to explicitly integrate more formal expert knowledge into the performance metric holds the most promise.

Acknowledgements

The research presented in this paper has received funding from the Bundesministerium für Bildung und Forschung in the Rob-aKademI project (project number 01IS20009C).

References

[1] F. T. Piller, Mass customization als wettbewerbsstrategisches konzept, in: Mass Customization: Ein wettbewerbsstrategisches Konzept im Informationszeitalter, DUV, Wiesbaden, 2006, pp. 153–236.

[2] E. Westkämper, Towards the Re-Industrialization of Europe, Springer Berlin, Heidelberg, 2014.

[3] W. Zou, M. Andulkar, U. Berger, Development of robot programming system through the use of augmented reality for assembly tasks, in: ISR 2018; 50th International Symposium on Robotics, VDE, 2018, pp. 1–7.

[4] Y. Fan, J. Luo, M. Tomizuka, A learning framework for high precision industrial assembly, in: 2019 International Conference on Robotics and Automation (ICRA), 2019, pp. 811–817.

[5] A. Lämmle, P. Tenbrock, B. Bálint, F. Nägele, W. Kraus, J. Váncza, M. F. Huber, Simulation-based learning of the peg-in-hole process using robot-skills, in: 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2022, pp. 9340–9346.

[6] A. Lämmle, Z. Xiang, B. A. Bálint, Extension of established modern physics simulation for the training of robotic electrical cabinet assembly, Procedia CIRP 107 (2022) 1317–1322.

[7] A. Lämmle, P. Rusch, W. Rust, M. Senneka, R. Awad, Analytical joining models for learning contact-rich cabinet assembly tasks from simulation, Procedia CIRP 104 (2021) 488–493.

[8] A. Lämmle, J. Krauß, R. Awad, Data driven joining models for simulation-based assembly learning, Procedia CIRP 104 (2021) 494–499.

[9] J. Ibarz, J. Tan, C. Finn, M. Kalakrishnan, P. Pastor, S. Levine, How to train your robot with deep reinforcement learning: lessons we have learned, The International Journal of Robotics Research 40 (4-5) (2021) 698–721.

[10] S.-J. Chung, N. Pollard, Predictable behavior during contact simulation: a comparison of selected physics engines, Computer Animation and Virtual Worlds 27 (3-4) (2016) 262–270.

[11] J.-B. Mouret, K. Chatzilygeroudis, 20 years of reality gap, in: Proceedings of the Genetic and Evolutionary Computation Conference Companion on - GECCO '17, ACM Press, New York, NY, USA, 2017, pp. 1121–1124.

[12] R. Kolbert, N. Chavan-Dafle, A. Rodriguez, Experimental validation of contact dynamics for in-hand manipulation, in: 2016 International Symposium on Experimental Robotics, Vol. 1, Springer International Publishing, 2017, pp. 633–645.

[13] P. C. Horak, J. C. Trinkle, On the similarities and differences among contact models in robot simulation, IEEE Robotics and Automation Letters 4 (2) (2019) 493–499.

[14] M. Kaspar, J. D. Muñoz Osorio, J. Bock, Sim2real transfer for reinforcement learning without dynamics randomization, in: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020, pp. 4383–4388.

[15] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, V. Vanhoucke, Sim-to-real: Learning agile locomotion for quadruped robots, CoRR abs/1804.10332 (2018).

[16] H.-n. Wang, N. Liu, Y.-y. Zhang, D.-w. Feng, F. Huang, D.-s. Li, Y.-m. Zhang, Deep reinforcement learning: a survey, Frontiers of Information Technology & Electronic Engineering 21 (12) (2020) 1726–1744.

[17] E. Todorov, T. Erez, Y. Tassa, Mujoco: A physics engine for model-based control, in: 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, IEEE, 2012, pp. 5026–5033.

[18] J. Serrà, J. L. Arcos, A competitive measure to assess the simlarity between two time series, in: B. D. Agudo, I. Watson (Eds.), Case-Based Reasoning Research and Development, Springer Berlin Heidelberg, Berlin, Heidelberg, 2012, pp. 414–427.

[19] L. Halt, F. Nagele, P. Tenbrock, A. Pott, Intuitive constraint-based robot programming for robotic assembly tasks, in: 2018 IEEE International Conference on Robotics and Automation (ICRA), 2018, pp. 520–526.

[20] F. Abu-Dakka, B. Nemec, A. Kramberger, A. Buch, N. Krüger, A. Ude, Solving peg-in-hole tasks by human demonstration and exception strategies, Industrial Robot: An International Journal 41 (2014) 575–584.

[21] C. C. Beltran-Hernandez, D. Petit, I. G. Ramirez-Alpizar, K. Harada, Variable compliance control for robotic peg-in-hole assembly: A deep-reinforcement-learning approach, Applied Sciences 10 (19) (2020) 6923.

[22] Z. Zhu, H. Hu, Robot learning from demonstration in robotic assembly: A survey, Robotics 7 (2) (2018).

[23] S.-k. Yun, Compliant manipulation for peg-in-hole: Is passive compliance a key to learn contact motion?, in: 2008 IEEE International Conference on Robotics and Automation, 2008, pp. 1647–1652.

[24] J. Serrà, J. L. Arcos, An empirical evaluation of similarity measures for time series classification, Knowledge-Based Systems 67 (2014) 305–314.

[25] P.-F. Marteau, Time warp edit distance with stiffness adjustment for time series matching, IEEE Transactions on Pattern Analysis and Machine Intelligence 31 (2) (2009) 306–318.

[26] D. J. Berndt, J. Clifford, Using dynamic time warping to find patterns in time series., in: KDD workshop, Vol. 10, Seattle, WA, USA:, 1994, pp. 359–370.

[27] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, W. Zaremba, Openai gym (2016).

[28] Y. Wu, E. Mansimov, S. Liao, R. B. Grosse, J. Ba, Scalable trust-region method for deep reinforcement learning using kronecker-factored approximation, CoRR abs/1708.05144 (2017).

[29] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, N. Dormann, Stable-baselines3: Reliable reinforcement learning implementations, Journal of Machine Learning Research 22 (268) (2021) 1–8.

[30] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, K. Kavukcuoglu, Asynchronous methods for deep reinforcement learning, in: Proceedings of The 33rd International Conference on Machine Learning, Vol. 48, PMLR, New York, New York, USA, 2016, pp. 1928–1937.

[31] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, CoRR abs/1707.06347 (2017).

[32] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: International conference on machine learning, 2018, pp. 1861–1870.